

Nama : Haidar Lathif Abqary Setia Harja

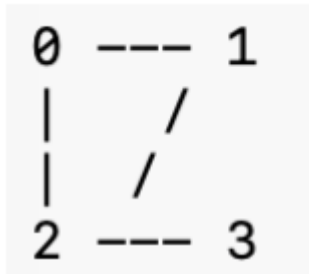
NIM : J0403251097

Kelas : A2

Praktikum 1

Graph:

Buat graph berikut:



Kode:

```
1  #Nama : Haidar Lathif Abqary Setia Harja
2  #NIM : J0403251097
3  #Kelas : A2
4  def matrixGraph(v, edges): #membuat fungsi pembuatan matriks
5      graph = [[0 for i in range(v)] for j in range(v)] #inisialisasi matriks
6
7      for u, v in edges: #mengisi matriks jika vertex bersambung
8          #vertex bersambung saling mengisi baris dan kolom dalam matriks
9          graph[u][v] = 1
10         graph[v][u] = 1
11     return graph
12
13 if __name__ == "__main__":
14     V = 4 #jumlah vertex
15     edges = [(1,2),(0,1),(0,2),(2,3)] #daftar edge antar vertex
16     graph = matrixGraph(V, edges) #memanggil fungsi
17     for row in range(V): #menampilkan isi baris matriks
18         for col in range(V): #menampilkan isi kolom per baris
19             print(graph[row][col], end=" ")
20         print()
```

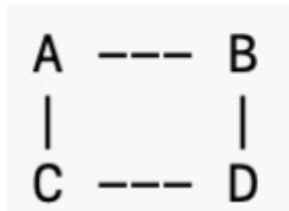
Output:

```
PS C:\Users\HP\Documents\Tugas (kuliah)\Semester 2\Algoritma dan Struktur Data> & C:/Users/HP/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/HP/Documents/Tugas (kuliah)/Semester 2/Algoritma dan Struktur Data/ASG_J0403251097/Praktikum11/Praktikum-1.py"
0 1 1 0
1 0 1 0
1 1 0 1
0 0 1 0
PS C:\Users\HP\Documents\Tugas (kuliah)\Semester 2\Algoritma dan Struktur Data> █
```

Praktikum 2

Graph:

Representasikan graph berikut:



Kode:

```
1  #Nama : Haidar Lathif Abqary Setia Harja
2  #NIM : J0403251097
3  #Kelas : A2
4  def ListGraph(v, edges): #membuat fungsi pembuatan list
5      graph = {i: [] for i in v} #inisialisasi list menggunakan dictionary
6      for u, v in edges: #mengisi list jika vertex bersambung
7          #vertex bersambung saling mengisi list
8          graph[u].append(v)
9          graph[v].append(u)
10     return graph
11
12 if __name__ == "__main__":
13     V = ["A", "B", "C", "D"] #daftar vertex
14     edges = [("A", "B"), ("A", "C"), ("B", "D"), ("C", "D")] #daftar edge antar vertex
15     graph = ListGraph(V, edges) #memanggil fungsi
16     for vertex in V: #menampilkan isi list untuk setiap vertex
17         print(f"{vertex}: {graph[vertex]}")
18
```

Output:

```
PS C:\Users\HP\Documents\Tugas (kuliah)\Semester 2\Algoritma dan Struktur Data> & C:/Users/HP/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/HP/Documents/Tugas (kuliah)/Semester 2/Algoritma dan Struktur Data/ASG_J0403251097/Praktikum11/Praktikum-2.py"
A: ['B', 'C']
B: ['A', 'D']
C: ['A', 'D']
D: ['B', 'C']
PS C:\Users\HP\Documents\Tugas (kuliah)\Semester 2\Algoritma dan Struktur Data> 
```

Praktikum 3

Graph:

```
matrix = [  
    [0,1,1,0],  
    [1,0,1,0],  
    [1,1,0,1],  
    [0,0,1,0]  
]
```

Kode:

```
1  #Nama : Haidar Lathif Abqary Setia Harja  
2  #NIM : J0403251097  
3  #Kelas : A2  
4  def matrixtolist(matrix): #membuat fungsi konversi  
5      graph = {} #inisialisasi list menggunakan dictionary  
6      for i in range(len(matrix)): #membaca setiap baris matriks  
7          graph[i] = [] #inisialisasi list untuk setiap vertex yang bersambung  
8          for j in range(len(matrix[i])): #membaca setiap kolom dalam baris matriks  
9              #mengubah isi matriks menjadi list jika vertex bersambung  
10             if matrix[i][j] == 1:  
11                 graph[i].append(j)  
12     return graph  
13  
14 if __name__ == "__main__":  
15     matrix = [  
16         [0, 1, 1, 0],  
17         [1, 0, 1, 0],  
18         [1, 1, 0, 1],  
19         [0, 0, 1, 0]  
20     ] #isi matriks yang akan dikonversi  
21     graph = matrixtolist(matrix) #menkonversi matriks menjadi list  
22     for vertex in graph: #menampilkan isi list untuk setiap vertex  
23         print(f"{vertex}: {graph[vertex]}")
```

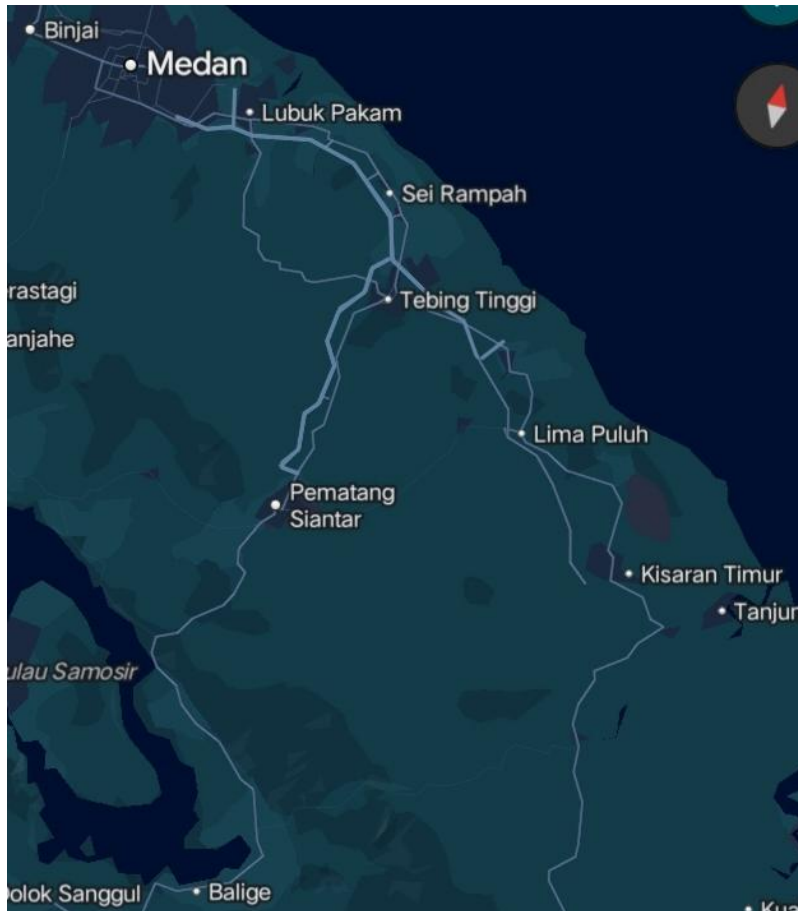
Output:

```
PS C:\Users\HP\Documents\Tugas (kuliah)\Semester 2\Algoritma dan Struktur Data> & C:/Users/HP/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/HP/Documents/Tugas (kuliah)/Semester 2/Algoritma dan Struktur Data/ASG_J0403251097/Praktikum11/Praktikum-3.py"  
0: [1, 2]  
1: [0, 2]  
2: [0, 1, 3]  
3: [2]  
PS C:\Users\HP\Documents\Tugas (kuliah)\Semester 2\Algoritma dan Struktur Data> []
```

Praktikum 4

Studi Kasus:

Saya memilih untuk membuat simulasi graph jalan antar kota besar di Sumatera Utara mulai dari Binjai sampai Balige atau Kisaran timur



Kode:

```
if __name__ == "__main__":
    kota = ["Binjai", "Medan", "Tebing Tinggi", "Pematang Siantar", "Lima Puluh", "Kisaran Timur", "Balige"]
    jalan = [("Binjai", "Medan"), ("Binjai", "Tebing Tinggi"), ("Medan", "Tebing Tinggi"), ("Tebing Tinggi", "Pematang Siantar"), ("Pematang Siantar", "Lima Puluh"), ("Lima Puluh", "Kisaran Timur"), ("Kisaran Timur", "Balige")]
    List_graph = JalanKotaList(kota, jalan) #memanggil fungsi pembuatan list
    Matrix_graph = JalanKotaMatrix(kota, jalan) #memanggil fungsi pembuatan matriks
    #menampilkan hasil
    print("Hubungan antar kota dalam bentuk matriks dan list adjacency:")
    print("\nMatriks Adjacency:")
    for row in range(len(kota)):
        for col in range(len(kota)):
            print(Matrix_graph[row][col], end=" ")
        print()

    print("\nList Adjacency:")
    for vertex in kota:
        print(f"{vertex}: {List_graph[vertex]}")
```

Matriks:

```
4 def JalanKotaMatrix(kota, jalan): #membuat fungsi pembuatan matriks
5     graph = [[0 for i in range(len(kota))] for j in range(len(kota))] #inisialisasi matriks
6     for a, t in jalan: #mengisi matriks jika kota bersambung
7         #mengkonversi nama kota menjadi index untuk mengisi matriks
8         index_a = kota.index(a)
9         index_t = kota.index(t)
10        #mengisi matriks menggunakan index kota yang bersambung
11        graph[index_a][index_t] = 1
12        graph[index_t][index_a] = 1
13    return graph
14
15 def JalanKotaList(kota, jalan): #membuat fungsi pembuatan list
16     graph = {i: [] for i in kota} #inisialisasi list menggunakan dictionary
17     for a, t in jalan: #mengisi list jika kota bersambung
18         #mengisi list dengan nama kota yang bersambung
19         graph[a].append(t)
20         graph[t].append(a)
21    return graph
```

List:

```
15 def JalanKotaList(kota, jalan): #membuat fungsi pembuatan list
16     graph = {i: [] for i in kota} #inisialisasi list menggunakan dictionary
17     for a, t in jalan: #mengisi list jika kota bersambung
18         #mengisi list dengan nama kota yang bersambung
19         graph[a].append(t)
20         graph[t].append(a)
21    return graph
```

Output:

Matriks:

```
Matriks Adjacency:
0 1 1 0 0 0 0
1 0 1 0 0 0 0
1 1 0 1 1 0 0
0 0 1 0 0 0 1
0 0 1 0 0 1 0
0 0 0 0 1 0 0
0 0 0 1 0 0 0
```

List:

List Adjacency:

Binjai: ['Medan', 'Tebing Tinggi']

Medan: ['Binjai', 'Tebing Tinggi']

Tebing Tinggi: ['Binjai', 'Medan', 'Pematang Siantar', 'Lima Puluh']

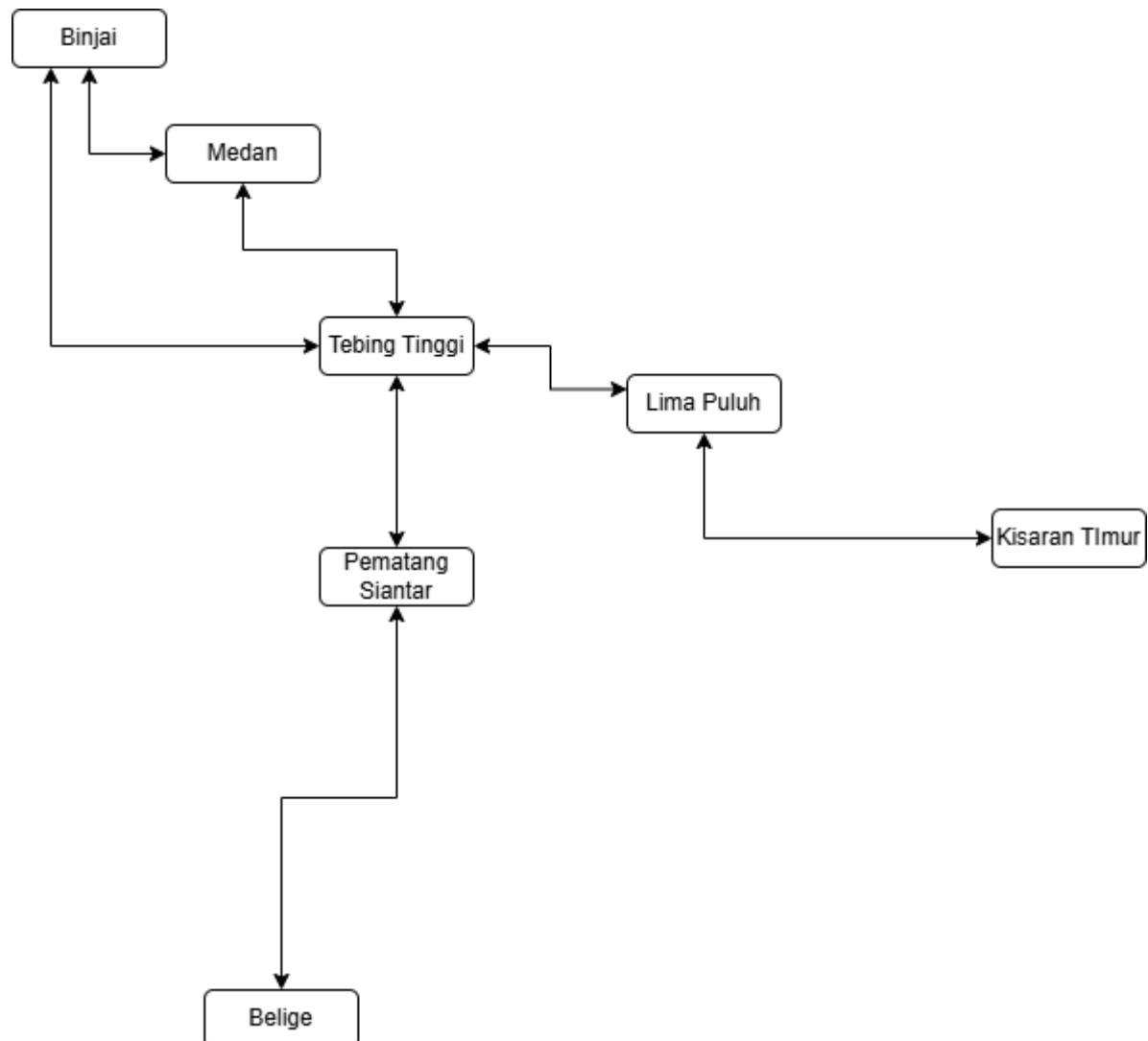
Pematang Siantar: ['Tebing Tinggi', 'Balige']

Lima Puluh: ['Tebing Tinggi', 'Kisaran Timur']

Kisaran Timur: ['Lima Puluh']

Balige: ['Pematang Siantar']

Desain Graph



Penjelasan Node dan Edge:

Kota digunakan sebagai nama dari node dan jalan digambarkan sebagai edge dari node/vertex.

Hubungan jalan antar kota bersifat dua arah (bolak balik) sehingga graph dapat dikategorikan berjenis tidak berarah (*undirected*).

Jumlah edge antar node juga tidak terlalu banyak sehingga graph dikategorikan sebagai parse graph

Analisis Singkat:

1. Jenis graph: Graph tidak berarah (*undirected graph*) karena hubungan antar kota bersifat dua arah.
2. Alasan memilih edge: merepresentasikan jalan asli antar kota yang ada di peta.
3. Representasi yang lebih cocok: List adjacency lebih cocok karena lebih efisien dalam menyimpan informasi dan hubungan antar kota cenderung linear (terhubung dalam satu jalan dua arah).
4. Kategori graph: Graph termasuk dalam kategori parse graph karena jumlah edge cenderung sedikit dibandingkan dengan jumlah vertex.

Kesimpulan Singkat:

Terdapat perbedaan antara implementasi data yang saling terhubung menggunakan graph matrix dan graph list. Graph matrix umumnya digunakan untuk data (node/vertex) yang memiliki hubungan (edge) yang banyak (*dense graph*). Sedangkan graph list umumnya digunakan untuk data yang memiliki hubungan yang relatif sedikit (*parse graph*). Keduanya memiliki kelebihan dan kekurangan masing-masing.